Curtin University

School of Electrical Engineering, Computing and Mathematical Sciences (EECMS)

COMP6006 Web Application Frameworks

Critical Analysis of Real-Time Web Application Design

Conference Connect

Prepared by:	F.M. Ashfaq
Student ID:	23472297
Student contact:	f.m.ashfaq@postgrad.curtin.edu.au
Unit:	COMP6006 – Web Application Frameworks
Submitted to:	Dr. Sheik Mohammad Mostakim Fattah
Date:	May 2026

Contents

Chapter 1

Introduction

This report critically analyses *Conference Connect*, a real-time conference web application developed for the COMP6006 Web Application Frameworks final assignment (Curtin University, 2026). The application was built as a React single page application with a Node.js and Express backend, Passport-based LinkedIn authentication, and Socket.IO real-time communication.

The purpose of this report is to evaluate the main technical design decisions rather than only describe the final product. The discussion focuses on SPA architecture, state management, routing, real-time communication, authentication, route protection, security limitations, and future improvements. The application includes protected login, conference session browsing, saved schedules, active live sessions, real-time chat, participant synchronisation, and live room counts.

Chapter 2

SPA Architecture and State Management

The frontend was implemented using React, Vite, React Router, and React-Bootstrap. React supports component-based user interface development, Vite provides the development and build environment, React Router manages client-side routing, and React-Bootstrap provides responsive interface components (Meta Open Source, 2026; React Bootstrap, 2026; React Router, 2026; Vite, 2026). This combination was appropriate because the application needed fast navigation between pages without full reloads.

The frontend is organised into pages, reusable components, context providers, and helper modules. Main views such as Login, Dashboard, My Schedule, Active Sessions, and Live Session are separated into page components. Shared interface elements, including the navigation bar, protected route wrapper, and session card, are placed in reusable components. This improves maintainability because common behaviour is centralised instead of repeated across multiple pages.

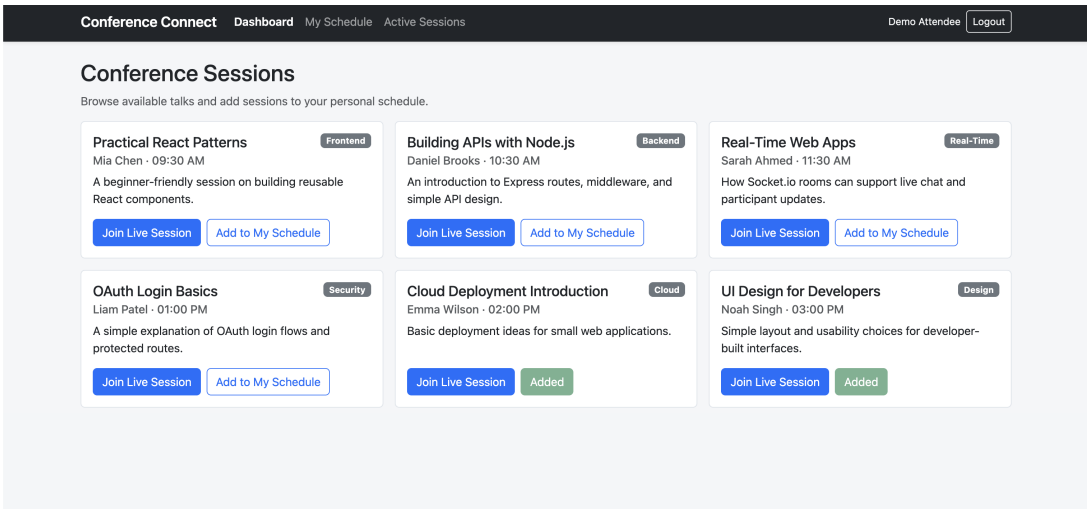

Figure 2.1: Dashboard page showing available conference sessions

State management was handled using React's built-in state and context rather than an external library such as Redux. `AuthContext` stores the current user and authentication state, while `ScheduleContext` stores selected session IDs and persists them through browser `localStorage`. This was a reasonable choice because the shared state is small and the assignment scope does not require complex global data handling.

The main advantage is simplicity. The schedule can be updated immediately on the client and remains available after a refresh. However, this design also has a limitation: the schedule belongs to the browser rather than the authenticated account. If the same user logs in from another device, the schedule will not follow them. A production version should store schedules in a database linked to the authenticated user.

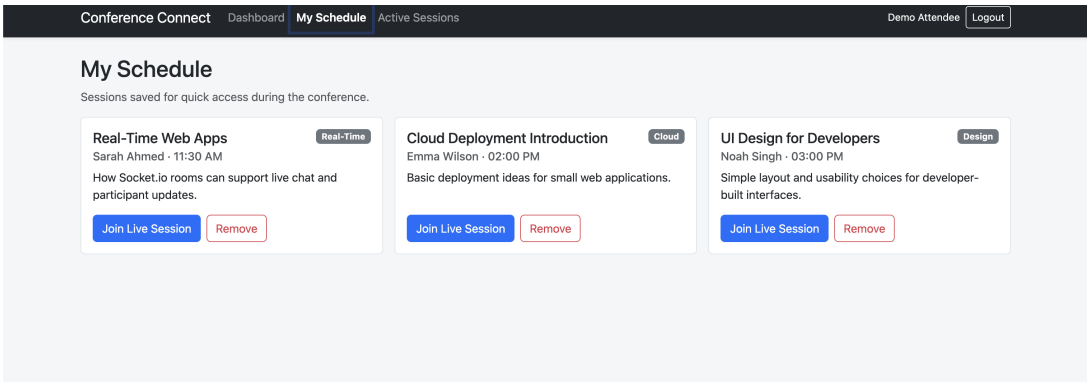

Figure 2.2: My Schedule page showing saved conference sessions

Overall, the SPA architecture is suitable for the assignment because it is clear, modular, and easy to demonstrate. Its main trade-off is that the state management approach is practical for a small project but not sufficient for a larger multi-device production system.

Chapter 3

Real-Time Communication Design

The real-time features were implemented using Socket.IO on the frontend and backend. Socket.IO supports low-latency, bidirectional, event-based communication, which matches the requirement for live conference rooms and chat (Socket.IO, 2026). Each conference session uses its session ID as the Socket.IO room name. When a user enters a live session, the frontend emits a `joinRoom` event with the session ID.

The backend stores active room data in memory. Each room has a participants map and a messages array. The participants map uses socket IDs, making it easy to remove users when they leave or disconnect. The messages array stores chat history while the server is running. This satisfies the assignment requirement for runtime chat history without adding database complexity.

The event design is intentionally simple. The client emits `joinRoom`, `sendMessage`, and `leaveRoom`. The server emits `chatHistory`, `chatMessage`, `participants`, `roomNotice`, and `activeRooms`. This keeps the real-time logic understandable while supporting room-specific chat, participant lists, and active room counts.

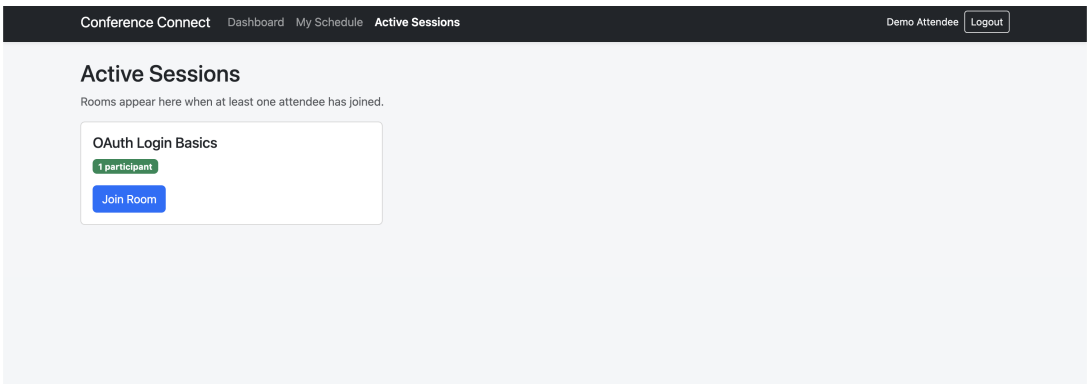

Figure 3.1: Active Sessions page showing live room status and participant counts

A major benefit of Socket.IO rooms is separation. A message sent in one live session is only delivered to that session's room, not to all connected users. Participant synchronisation is

also handled on the server, which is better than letting each client guess who is currently connected. When a user joins, leaves, or disconnects, the server updates the room state and emits the latest participant list.

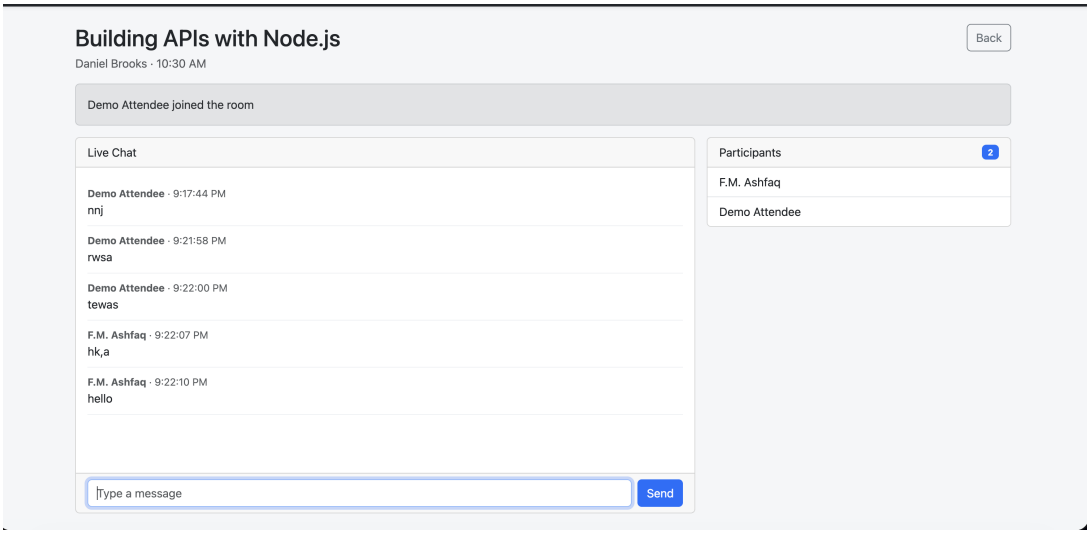

Figure 3.2: Live Session page showing real-time chat and participant synchronisation

The main limitation is scalability. Because all room data is stored in one Node.js process, chat history and participant state are lost when the server restarts. It would also be difficult to scale across multiple backend instances without shared state. A production version should use persistent message storage and a shared Socket.IO adapter such as Redis.

Chapter 4

Authentication and Security Considerations

Authentication is implemented using Passport.js with LinkedIn OAuth. Passport provides authentication middleware for Node.js applications, while Express provides the backend web framework used for routes, sessions, and API handling (Express.js, 2026; Passport.js, 2026). After successful LinkedIn login, the backend stores a simplified user object in the Express session. The frontend checks the current authenticated user through the `/auth/user` endpoint.

The login page provides the entry point for authenticated use of the application. Protected pages are not intended to be available until a user has logged in. The project also includes a local demo login for easier testing with multiple users during development. This is useful for demonstrating chat and participant counts but should not be treated as a production authentication method.

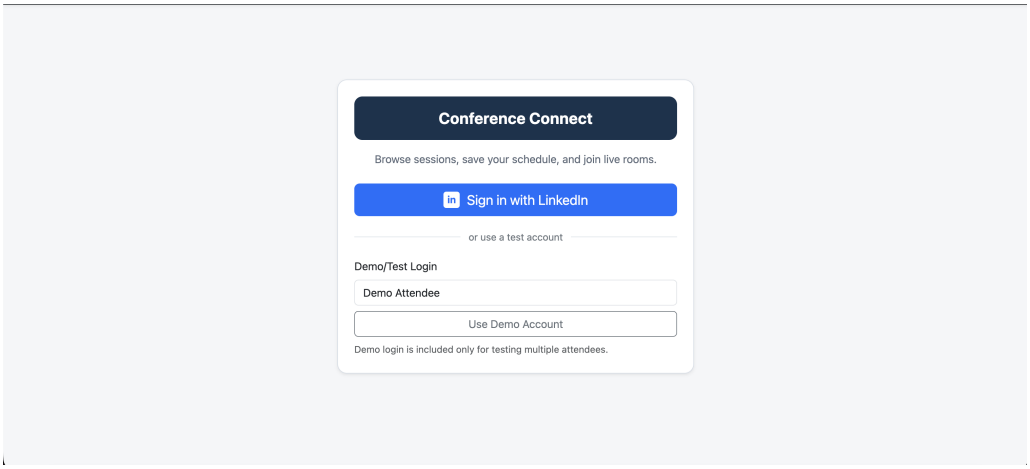

Figure 4.1: Login page with LinkedIn authentication and demo login option

Route protection is implemented on both the frontend and backend. On the frontend, `ProtectedRoute` redirects unauthenticated users to the login page. On the backend, protected

API routes require an authenticated session. Socket.IO also checks the Express session before allowing a socket to join live rooms, which prevents unauthenticated users from sending messages through the normal application flow.

The design has useful security strengths. LinkedIn credentials are stored in environment variables, the real `.env` file is excluded from version control, and session cookies are HTTP-only. However, several production protections are simplified. The application uses an in-memory session store, local HTTP settings, no full CSRF protection, no rate limiting, and only basic message trimming. For production, HTTPS, secure cookies, a persistent session store, stronger validation, and abuse prevention would be required.

Chapter 5

Critical Reflection and Future Improvements

The most challenging part of the implementation was connecting Express session authentication with Socket.IO. The application had to authenticate users through HTTP first, then allow the same authenticated session to access real-time socket rooms. This required sharing session logic between Express routes and Socket.IO middleware.

Another challenge was balancing simplicity and completeness. The application needed multiple rooms, chat history, participant lists, active room counts, and join or leave notices. I kept the implementation in memory because it matched the assignment scope and made the system easier to explain. The trade-off is that live data is temporary and not suitable for production.

Several design decisions were deliberate. React Context was used instead of Redux because the state requirements were small. React-Bootstrap was used to create a consistent interface quickly. The schedule was kept in `localStorage` to demonstrate frontend persistence without adding a database. The demo login was included to support testing, but LinkedIn remained the main authentication pathway.

Future improvements would focus on persistence, scalability, and security. A database should store users, schedules, sessions, and messages. A Redis-backed Socket.IO adapter would allow multiple backend instances to share room events. Security should be improved through HTTPS-only cookies, CSRF protection, rate limiting, stronger input validation, and moderation controls. The frontend could also be improved with better filtering, error pages, reconnection handling, and automated tests.

Chapter 6

Conclusion

Conference Connect demonstrates the main concepts required for the COMP6006 final assignment. It uses React SPA architecture, client-side routing, reusable components, context-based state management, Passport authentication, protected routes, and Socket.IO real-time communication.

The frontend design is maintainable because pages, components, and context providers are clearly separated. The real-time design is effective for a single-server demonstration because room membership, chat history, participant lists, and active room counts are handled through clear Socket.IO events. The authentication system demonstrates OAuth login, Express sessions, route protection, and authenticated socket access.

The main limitations are related to production readiness. The application uses browser-based schedule storage, in-memory room state, a non-production session store, and simplified security controls. These choices are acceptable for the assignment but would need to evolve for a real deployment. Overall, the project provides a functional and explainable real-time conference platform while showing the trade-offs involved in web application framework design.

References

Curtin University. (2026). COMP6006 Final Assignment (Part B): Critical Analysis of Real-Time Web Application Design.

Express.js. (2026). *Express documentation*. Retrieved May 28, 2026, from https://expressjs.com/

Meta Open Source. (2026). *React documentation*. Retrieved May 28, 2026, from https://react.dev/

Passport.js. (2026). *Passport documentation*. Retrieved May 28, 2026, from https://www.passportjs.org/docs/

React Bootstrap. (2026). *React bootstrap documentation*. Retrieved May 28, 2026, from https://react-bootstrap.netlify.app/

React Router. (2026). *React router documentation*. Retrieved May 28, 2026, from https://reactrouter.com/

Socket.IO. (2026). *Socket.io documentation*. Retrieved May 28, 2026, from https://socket.io/docs/v4/

Vite. (2026). *Vite documentation*. Retrieved May 28, 2026, from https://vite.dev/